Modules: Classes & Objects (Constructors), Inheritance, and Abstract Classes.

Module: Classes, Objects and Constructors

Problem 1: E-Commerce Package Volume Calculator

Scenario

An e-commerce company needs to determine the storage space required for packages before shipping them. Each package is represented by its **length, width, and height**.

Problem Statement

Design a Java class named **Box** to represent a shipping package.

The class should:

- Store the **length**, **width**, and **height** (double values) using a parameterized constructor.
- Provide a method named **calculateVolume()** to compute the package volume.
- Create one or more `Box` objects and display the volume of each package.

Expected Outcome
Students will learn parameterized constructors, object creation, encapsulation, and instance methods.

Problem 2: Scientific Calculator Utility

Scenario

A scientific calculator application frequently performs exponential calculations such as compound growth and scientific computations. Since these calculations are independent of any specific calculator object, they should be implemented as utility methods.

Problem Statement

Develop a Java utility class named **Calculator** containing the following static methods:

- `powerInt(int base, int exponent)`
- `powerDouble(double base, int exponent)`

Both methods should return the value of **base raised to the specified exponent**.

Invoke these methods directly using the class name without creating an object.

Hint: Use `Math.pow()`.

Expected Outcome
Students will understand static methods, utility classes, and method overloading.

Problem 3: Hospital Patient Management System

Scenario

A hospital maintains basic health records for every patient. During registration, the hospital records the patient's name, height, and weight to estimate the patient's Body Mass Index (BMI).

Problem Statement

Design a Java class named **Patient** that stores:

- Patient Name
- Height
- Weight

Implement a method named **calculateBMI()** that computes and returns the patient's Body Mass Index (BMI).

Create another class named **HospitalManagement** to:

- Create one or more Patient objects.
- Display the patient's details.
- Display the calculated BMI.

Expected Outcome
Students will learn class design, constructors, object interaction, and encapsulation.

Module: Inheritance

Problem 1: Wildlife Information System

Scenario

A wildlife sanctuary maintains information about different animals. Every animal can eat and sleep, whereas birds can additionally fly.

Problem Statement

Design a superclass named **Animal** containing methods:

- `eat()`
- `sleep()`

Create a subclass named **Bird** that:

- Overrides both methods.
- Introduces an additional method named `fly()`.

Create objects of both classes and demonstrate inheritance and method overriding.

Expected Outcome
Understand inheritance, method overriding, and runtime behavior.

Problem 2: Human Resource Management System

Scenario

An organization stores common employee details such as name, while payroll information is maintained separately.

Problem Statement

Develop:

- A superclass **Person** containing employee name.
- A subclass **Employee** containing:
  - Annual Salary
  - Joining Year
  - National Insurance Number

Provide suitable constructors, getter methods, and display methods.

Create a **TestEmployee** class to demonstrate inheritance.

Expected Outcome
Implement inheritance for extending existing classes.

Problem 3: University Management System

Scenario

A university maintains records for students, teachers, and college students. Although they all share common personal information, each category stores additional specialized information.

Problem Statement

Create the following class hierarchy:

```
Person
     |
```

```
 --------------------
 |                  |
Student          Teacher
 |
CollegeStudent
```

Requirements:

Person

- Name
- Age
- Address

Student

- Roll Number

Teacher

- Subject
- Salary

CollegeStudent

- Academic Year
- Major Specialization

Create suitable constructors and display methods.

Expected Outcome
Learn multilevel inheritance and hierarchical inheritance.

Problem 4: Fruit Information System

Scenario

A fruit delivery application displays different information depending on the type of fruit selected by the customer.

Problem Statement

Create a superclass named **Fruit** containing:

- Name
- Taste
- Size

Define an `eat()` method.

Create subclasses:

- Apple
- Orange

Override the `eat()` method to display information specific to each fruit.

Expected Outcome
Implement method overriding.

Problem 5: Drawing Software

Scenario

A computer graphics application supports multiple geometric shapes. Every shape can be drawn and erased, but the implementation differs depending on the shape.

Problem Statement

Develop a superclass **Shape** containing:

- `draw()`
- `erase()`

Create subclasses:

- Circle
- Triangle
- Square

Override both methods appropriately.

Create objects using superclass references to demonstrate runtime polymorphism.

Expected Outcome
Understand dynamic method dispatch and polymorphism.

Module: Abstract Classes

Problem 1: Banking Interest Rate Management System

Scenario

A financial comparison portal displays the savings and fixed deposit interest rates offered by different banks. Every bank provides these rates differently, but all banks follow the same interface.

Problem Statement

Create an abstract class named **GeneralBank** containing abstract methods:

- `getSavingInterestRate()`
- `getFixedDepositInterestRate()`

Create two subclasses:

- ICICIBank
- KotMBank

Override the methods to return their respective interest rates.

Create different object references to observe runtime polymorphism.

Expected Outcome
Understand abstract classes and abstraction.

Problem 2: Indian Railway Coach Management

Scenario

Indian Railways categorizes coaches into different compartments. Each compartment displays different passenger instructions.

Problem Statement

Create an abstract class **Compartment** containing an abstract method:

`notice()`

Develop subclasses:

- FirstClass
- Ladies
- General
- Luggage

Each subclass should display an appropriate notice.

Create an array of compartments and populate it with randomly generated compartment objects to demonstrate polymorphism.

Expected Outcome
Learn abstract classes, polymorphism, and arrays of objects.

Problem 3: Digital Music Studio

Scenario

A digital music application supports multiple musical instruments. Although every instrument can be played, each produces a different sound.

Problem Statement

Create an abstract class **Instrument** containing an abstract method:

`play()`

Develop subclasses:

- Piano
- Guitar
- Flute

Each subclass should override the `play()` method to display the sound produced by the instrument.

Store different instrument objects in an array of `Instrument` references.

Use the `instanceof` operator to identify the actual object stored at each index.

Expected Outcome
Students will understand abstraction, polymorphism, runtime object identification, and heterogeneous collections.